Architecture Réseaux Entreprises (ARES)

Data plane, segmentation et policy enforcement sous Linux

Brice - Ekane (brice.ekane@univ-rennes.fr)

ISTIC Rennes - France

2025-2026

git clone https://github.com/bekane/ares-2025.git

Plan du module

1. Objectifs et introduction
2. Chemin paquet et dataplane
3. Forwarding et policy routing
4. Redondance, chemins multiples et segmentation
5. Policy enforcement

Plan

1 Objectifs et introduction

Objectifs pédagogiques

- Comprendre Linux comme **dataplane** : interfaces, FIB, voisinage, hooks Netfilter.
- Expliquer le chemin réel d'un paquet : `ip rule`, tables, LPM, nexthop, ECMP, ARP/NDP.
- Concevoir une segmentation avec namespaces, VRF, tables dédiées, NAT et filtrage stateful.
- Diagnostiquer les effets d'une policy : asymétrie, `conntrack`, `rp_filter` et retour de flux.

Du paquet au dataplane Linux

- Le forwarding consiste à sélectionner un **nexthop** et une interface de sortie pour un paquet IP.
- La décision dépend du préfixe le plus spécifique, mais aussi des règles de policy routing et du contexte du paquet.
- Le dataplane Linux rend cette décision observable avec `ip route`, `ip rule`, `ip route get`, `nft` et `conntrack`.

Carte de la pile réseau Linux



Objets observables

Interfaces et adresses, règles de sélection, tables de routage, voisinage, état des flux et politique de filtrage participent au chemin d'un paquet.

Fil directeur

Chaque mécanisme du module modifie un objet de cette pile ou sa frontière d'isolation ; le diagnostic consiste à identifier lequel.

Fil rouge : segmenter sans perdre l'exploitabilité

Scénario

Plusieurs zones partagent un dataplane Linux, sans partager les mêmes chemins ni les mêmes droits.

- Dataplane : retrouver le chemin réel.
- Segmentation : séparer piles et tables.
- Policy : autoriser, traduire et tracer.
- Preuve : route, règles, état et capture.

Du besoin métier au dataplane

Point de départ

Partir du besoin, puis choisir le mécanisme minimal.

- **Isolation** : quels contextes séparer ?
- **Contrôle** : où appliquer la politique ?
- **Symétrie** : par où revient le flux ?
- **Preuve** : comment vérifier le comportement ?

Choisir un mécanisme de segmentation

Séparer

- **VLAN** : domaine L2
- **Sous-réseau** : frontière L3
- **VRF** : table de routage
- **Namespace** : pile complète

Contrôler

- **NAT** : traduire ou publier
- **nftables** : filtrer le chemin réel

Règle du module

Un mécanisme répond à un flux, une contrainte et une preuve.

Rappels L2/L3 utiles au dataplane

- Ethernet transporte une trame vers un voisin L2 identifié par une adresse MAC.
- IP transporte un paquet vers une destination logique, possiblement hors lien local.
- Le routeur décrémente le TTL/Hop Limit, choisit un nexthop, puis réécrit l'en-tête L2.
- La décision de forwarding ne modifie pas l'adresse IP de destination ; le NAT est un traitement séparé.

Control plane acquis, dataplane observé

Control plane

- Les protocoles dynamiques peuvent être portés par FRR, un contrôleur SDN ou une plateforme Cloud.
- Leur rôle est d'installer des routes et des politiques dans le système.

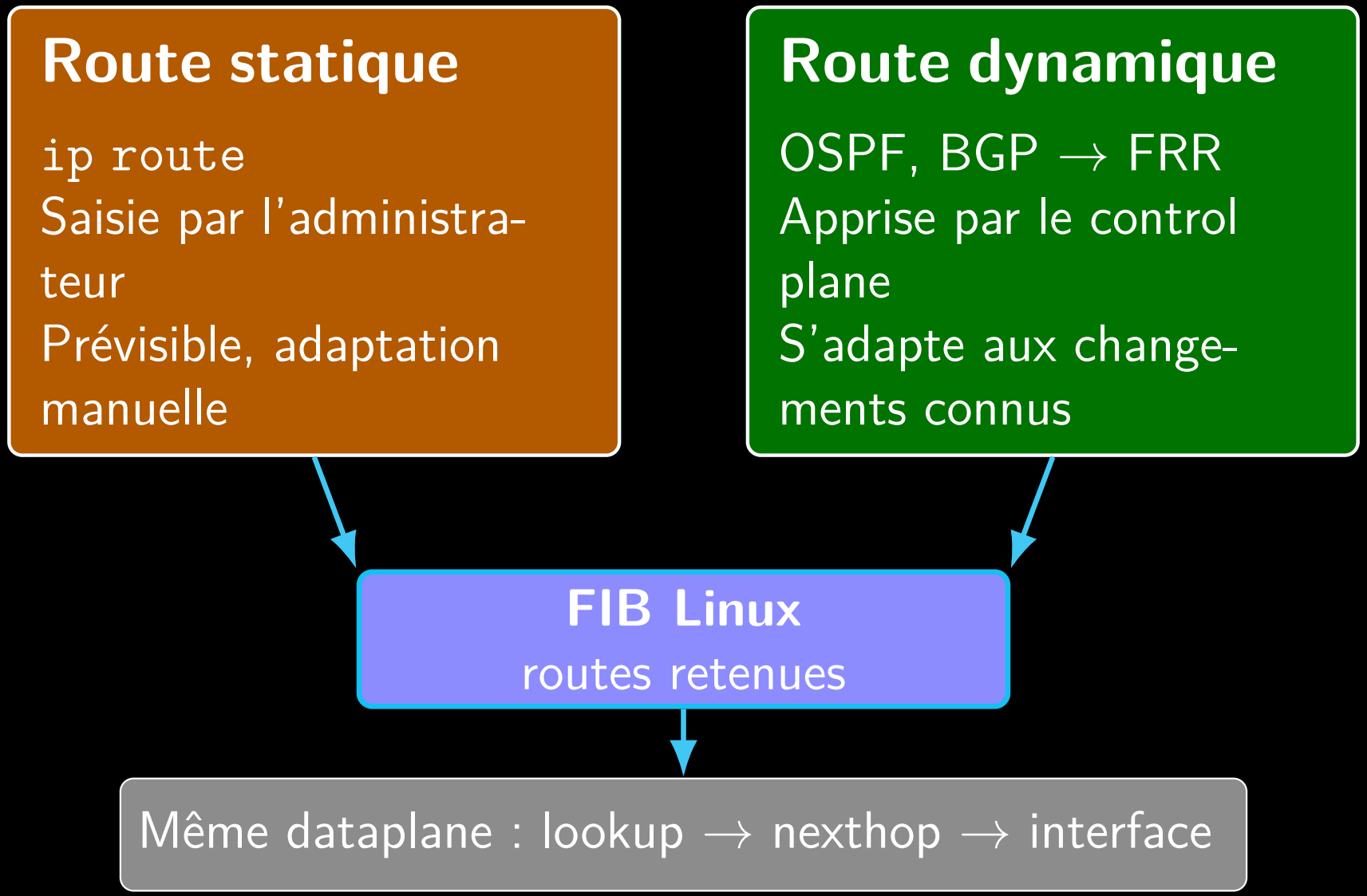
Dataplane

- Le noyau Linux applique ensuite les décisions paquet par paquet.
- C'est ce niveau que l'on manipule dans ce module.

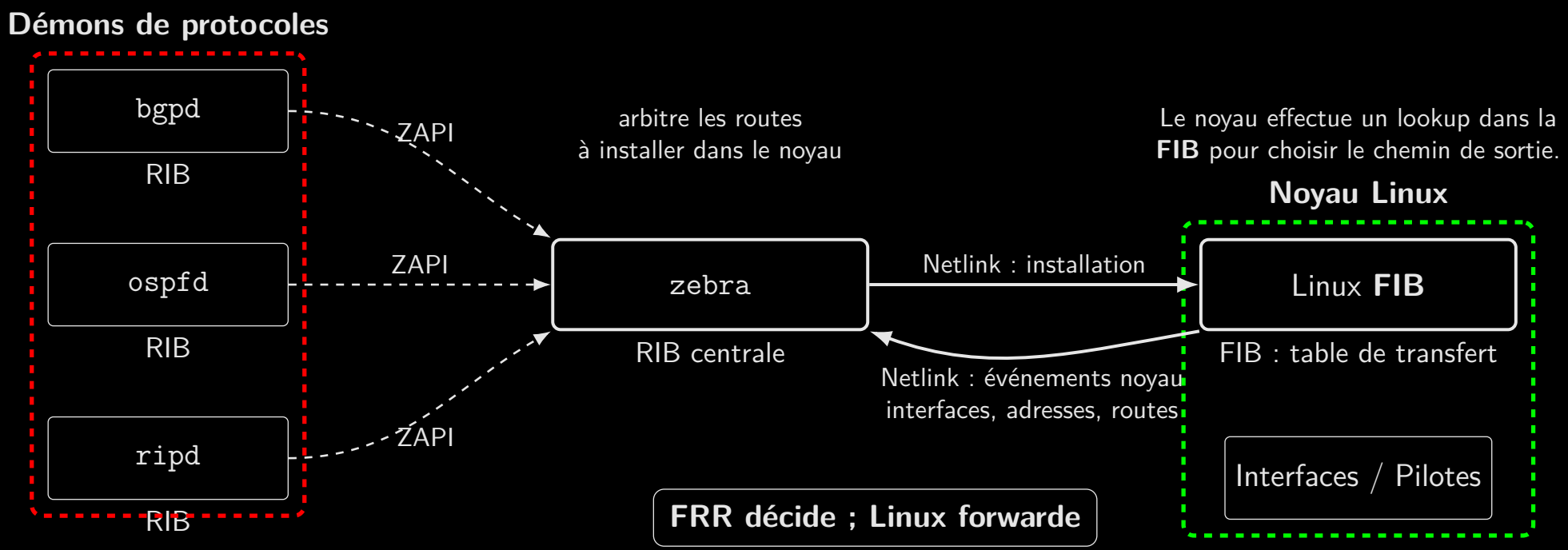
Plan

2 Chemin paquet et dataplane

Routage statique et dynamique : rappel



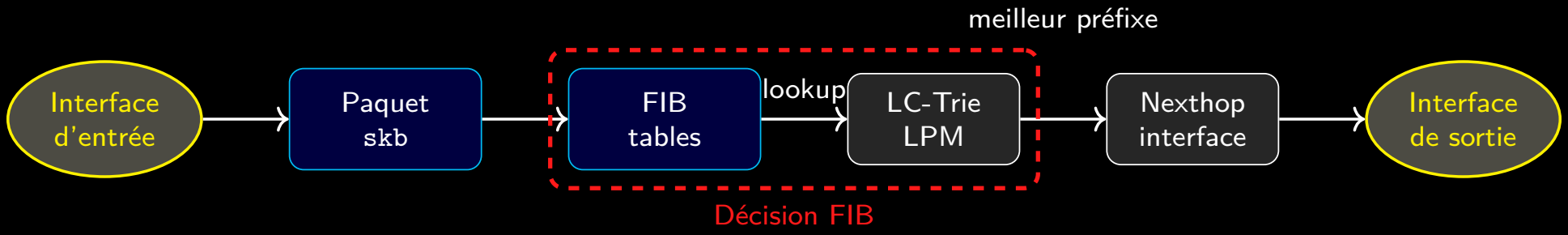
Control plane vers dataplane Linux



Que fait Linux quand un paquet arrive ?

Sélection du meilleur chemin

- La **FIB** (Forwarding Information Base) contient les routes retenues pour atteindre les destinations.
- C'est la FIB qui est utilisée pour faire avancer le paquet.



Longest Prefix Match : choisir la route la plus précise

Routes disponibles

- `0.0.0.0/0`
- `10.0.0.0/8`
- `10.1.0.0/16`
- `10.1.2.0/24`

Destination `10.1.2.3`

Les quatre préfixes correspondent, mais `10.1.2.0/24` est le plus spécifique : c'est cette route qui est choisie.

À retenir

Linux choisit le préfixe correspondant le plus long, pas la route écrite en premier ni celle dont la passerelle semble la plus proche.

Agrégation CIDR et résumés de routes

Pourquoi résumer ?

Plusieurs `10.20.x.0/24` peuvent devenir un résumé `10.20.0.0/16`.

- Le résumé suit le plan d'adressage et ses frontières.
- La route la plus spécifique reste prioritaire (LPM).
- Un résumé large nécessite une route de rejet locale.

Preuve

Tester une destination présente, puis une destination couverte mais absente.

Pourquoi le lookup reste rapide ?

Idée importante

Linux applique le LPM dans une FIB optimisée ; il ne parcourt pas naïvement toutes les routes pour chaque paquet.

- **CIDR / agrégation** : moins de routes installées et annoncées.
- **FIB optimisée** : tries ou LC-trie accélèrent le LPM.
- **Nexthop / voisinage séparés** : décision IP, puis ARP/NDP.
- **ECMP par hash** : choix stable par flux.
- **Offload possible** : routes ou règles poussées vers la carte ou l'ASIC.

À retenir

Performance = plan CIDR cohérent + structures optimisées du data-plane.

Plan

3. Forwarding et policy routing

Activer et vérifier le rôle de transit

```
# Activer temporairement le forwarding IPv4
sudo sysctl -w net.ipv4.ip_forward=1

# Interfaces et adresses réellement actives
ip -br address

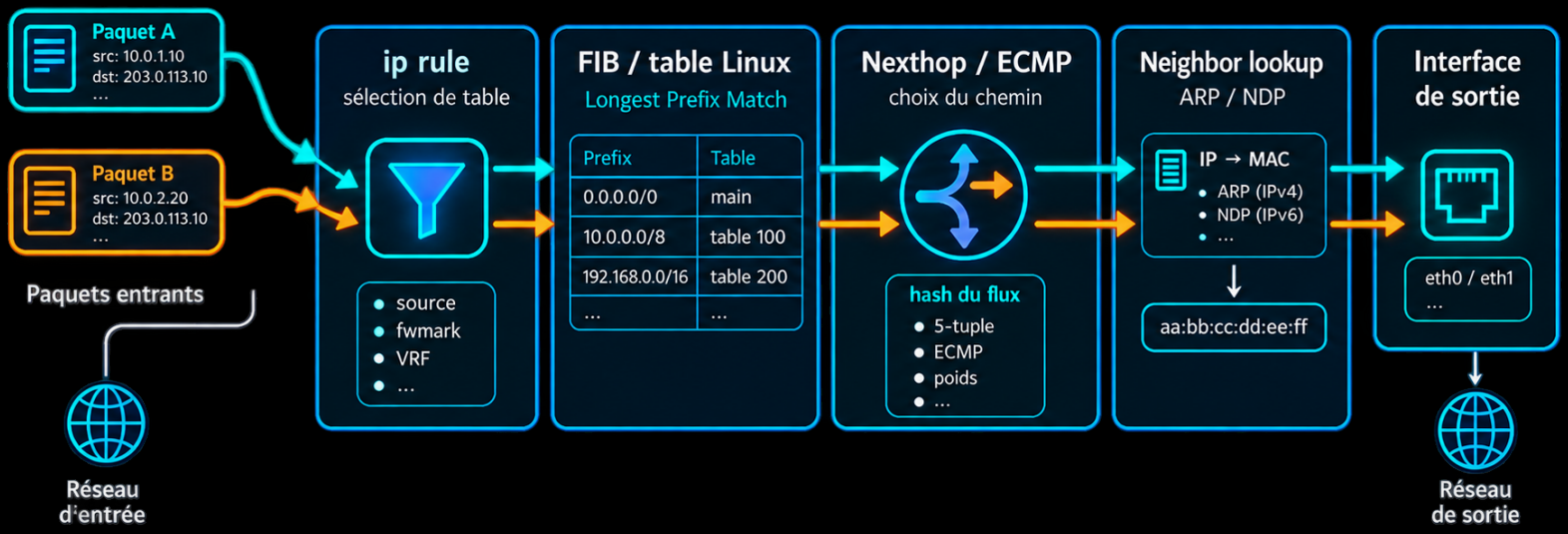
# Règles et routes réellement consultées
ip rule show
ip route show table all

# Rôle de transit effectivement activé
sysctl net.ipv4.ip_forward
```

Principe

Activer le transit est une décision d'architecture ; l'état du noyau en constitue la première preuve.

Décision de forwarding Linux



❓Pourquoi deux paquets vers la même destination peuvent-ils utiliser deux tables ou deux nexthops différents ?

Observer une décision de forwarding

```
# Règles de sélection des tables
ip rule show

# Tables connues par iproute2
cat /etc/iproute2/rt_tables

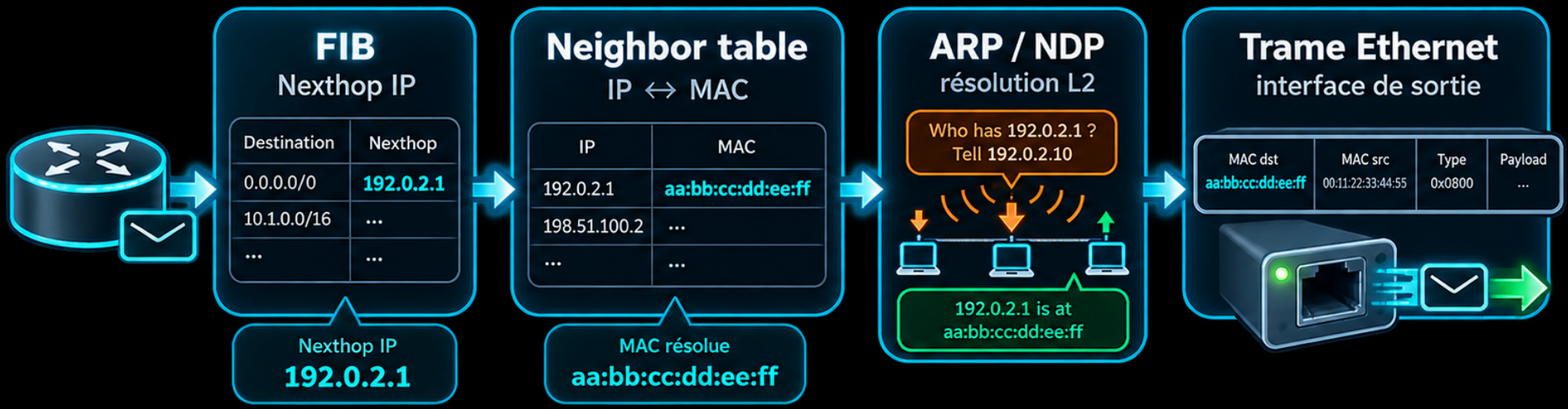
# Décision complète pour un paquet hypothétique
ip route get 8.8.8.8 from 10.10.10.42 iif eth1

# Voisinage L2 utilisé après le choix du nexthop
ip neigh show
```

Point clé

La table FIB ne suffit pas toujours : Linux consulte d'abord les règles, puis applique le Longest Prefix Match dans la table sélectionnée.

Neighbor table : une route correcte ne suffit pas



États utiles

REACHABLE
Entrée valide, utilisable

STALE
Entrée connue, à vérifier

INCOMPLETE
Résolution en cours

FAILED
Résolution échouée

Preuve

```
$ ip neigh show
192.0.2.1 dev eth0
  lladdr aa:bb:cc:dd:ee:ff
  REACHABLE
```

Diagnostic

Route et nexthop corrects
+ ARP/NDP en échec =
aucun paquet transmis.

Policy-based routing

Principe

La décision Linux ne dépend pas seulement de la destination. Les règles `ip rule` peuvent sélectionner une table selon la source, l'interface d'entrée, une marque firewall ou d'autres critères.

Cas Cloud/Réseau

- `LAN-A` sort par `ISP1`.
- `LAN-B` sort par `ISP2`.
- Le retour doit rester symétrique pour éviter les surprises avec NAT et conntrack.

Policy routing - exemple multi-uplink

```
# Noms de tables dans /etc/iproute2/rt_tables :
# 100 isp1
# 200 isp2

ip route add default via 203.0.113.1 dev eth0 table isp1
ip route add default via 198.51.100.1 dev eth1 table isp2

ip rule add from 10.10.10.0/24 table isp1 priority 100
ip rule add from 10.20.20.0/24 table isp2 priority 110

ip route get 1.1.1.1 from 10.10.10.42
ip route get 1.1.1.1 from 10.20.20.42
```

Reverse Path Filtering : contrôler la source

Question posée à la réception sur `eth1`

Pour une source `10.10.10.5`, par quelle interface Linux choisirait-il le meilleur chemin de retour ?

`rp_filter = 0`

Désactivé : aucun contrôle de chemin retour.

`rp_filter = 1`

Strict : le meilleur retour doit utiliser l'interface d'entrée.

`rp_filter = 2`

Loose : une route vers la source doit exister.

Conséquence architecturale

Multi-uplink / policy routing / ECMP → asymétrie possible → mode strict → paquet potentiellement rejeté.

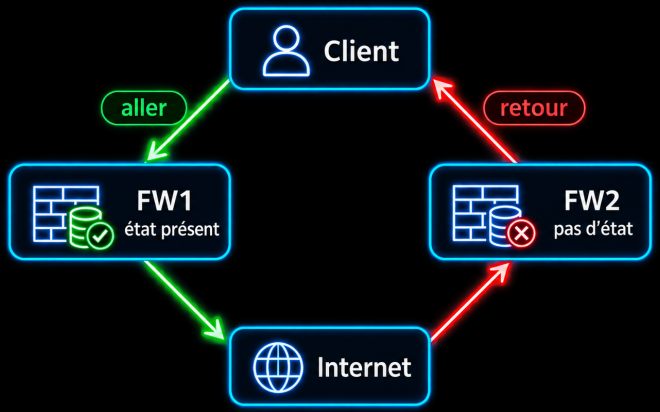
Policy routing - questions d'analyse

- Que se passe-t-il si un flux sort par `ISP1` mais revient par `ISP2` ?
- Quel est l'effet de `rp_filter` sur un trafic asymétrique ?
- Où faut-il appliquer le NAT pour préserver la symétrie ?
- Comment vérifier la table réellement utilisée par un paquet ?

À retenir

Le policy routing est souvent le vrai mécanisme derrière les architectures multi-uplink, multi-tenant et VPN.

Flux aller $\neq$ flux retour : où est l'état ?



Stateless

Le routage peut accepter deux chemins différents.

Stateful

NAT et conntrack attendent généralement que le retour retrouve l'état créé à l'aller ; `rp_filter` peut aussi intervenir.

Preuve attendue

Identifier le chemin dans les deux sens et le nœud qui porte l'état.

Preuves minimales d'un design dataplane

être capable de prouver

- La bonne table est sélectionnée pour chaque zone.
- Le nexthop choisi correspond au chemin attendu.
- Le NAT est appliqué au bon hook et dans le bon sens.
- Le retour de flux repasse par le bon état `conntrack`.
- Le diagnostic reste possible sans deviner le comportement du noyau.

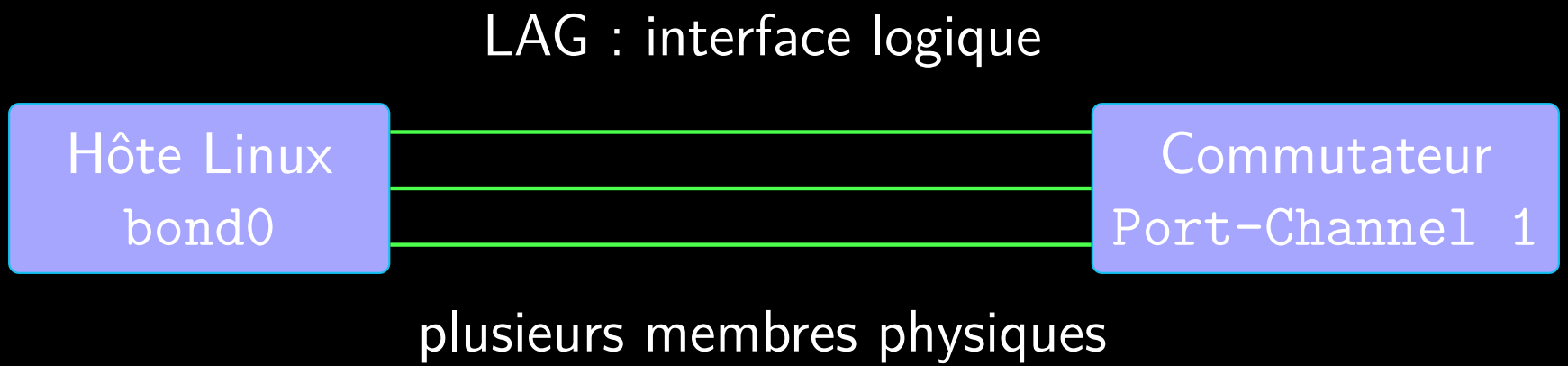
Commandes de validation

`ip rule`, `ip route get`, `ip neigh`, `nft list ruleset`, `conntrack -L`, `tcpdump`.

Plan

4 Redondance, chemins multiples et segmentation

LACP (1/4) : construire un lien logique



LAG et LACP

Le LAG forme l'interface logique ; LACP négocie et maintient ses membres physiques entre les deux équipements.

Condition

Les deux extrémités doivent agréger les mêmes liens avec une configuration cohérente.

LACP (2/4) : négocier les membres du LAG

Actor / Partner

- échange d'identité et de ports ;
- **active** initie, **passive** répond ;
- au moins une extrémité active.

Membre sélectionnable

- même agrégat aux deux extrémités ;
- vitesse, VLAN et MTU cohérents ;
- synchronisation et seuil *minimum-links*.

À retenir

LACP sélectionne les ports capables de former un même agrégat.

LACP (3/4) : hash, panne et preuve

Répartition

- un hash choisit le membre par flux ;
- un flux unique ne cumule pas les débits.

Défaillance

- le membre défaillant est retiré ;
- les flux sont redistribués si le seuil reste atteint.

Preuve attendue

Couper un membre ; mesurer perte, convergence et débit.

LACP (4/4) : implémentation sous Linux

```
# Créer un bond utilisant IEEE 802.3ad / LACP
ip link add bond0 type bond mode 802.3ad

# Rattacher les interfaces physiques
ip link set eno1 master bond0
ip link set eno2 master bond0
ip link set bond0 up

# Prouver le mode, l'agrégateur et l'état des membres
ip -d link show bond0
cat /proc/net/bonding/bond0
```

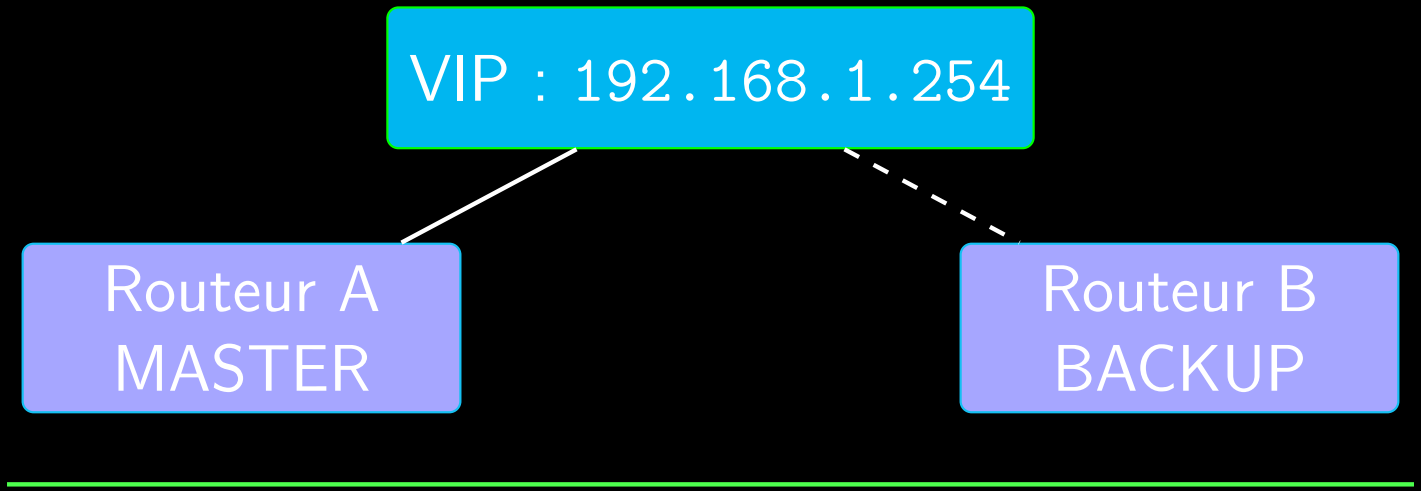
Côté Linux

Le pilote bonding porte bond0, exécute LACP et applique la politique de hash.

Côté commutateur

Les ports correspondants doivent appartenir au même LAG en mode LACP.

VRRP (1/5) : une passerelle virtuelle



Abstraction fournie

Les hôtes configurent une IP de passerelle stable. Un seul routeur porte normalement la VIP et assure le forwarding à un instant donné.

Périmètre

VRRP protège la passerelle virtuelle, pas le service de bout en bout.

VRRP (2/5) : élection et bascule

Choisir le MASTER

- VRID commun ;
- priorité et annonces déterminent le rôle ;
- préemption optionnelle.

Reprendre la VIP

- annonces absentes : bascule ;
- ARP gratuit / NDP actualise les voisins ;
- routes et politiques identiques.

Effet observable

La VIP change de porteur ; la passerelle configurée ne change pas.

VRRP (3/5) : implémentation sous Linux

Control plane en espace utilisateur

Un démon pilote l'élection et la VIP ; le noyau transfère les paquets.

Keepalived

- solution courante sous Linux ;
- suivi d'interfaces et de services.

FRR `vrrpd`

- VRRPv2 et VRRPv3 ;
- intégré au control plane FRR.

Même dataplane

Keepalived ou FRR pilote la VIP ; le dataplane reste celui du noyau.

VRRP (4/5) : exemple avec Keepalived

```
# /etc/keepalived/keepalived.conf
vrrp_instance VI_1 {
    state BACKUP
    interface eth0
    virtual_router_id 51
    priority 100
    advert_int 1
    virtual_ipaddress {
        192.168.1.254/24
    }
}
```

Déployer et valider

Même VRID et VIP sur les deux nœuds, mais priorités différentes. Démarrer Keepalived, puis identifier le porteur avec `ip address show eth0`.

VRRP (5/5) : bascule de VIP $\neq$ continuité du service

Ce que VRRP reprend

- la VIP et le rôle MASTER ;
- la passerelle des nouveaux flux.

Ce que VRRP ne réplique pas

- `conntrack` et NAT actif ;
- sessions applicatives.

Preuve et question M2

Mesurer la VIP, un nouveau flux et une connexion déjà établie.

Namespace réseau (1/5) : une pile indépendante

ns-prod

`eth0` – `10.0.0.1/24`

- FIB et règles propres ;
- voisinage et Netfilter propres ;
- sockets des processus attachés à `ns-prod`.

ns-test

`eth0` – `192.168.0.1/24`

- autre FIB, autres règles ;
- interfaces et adresses distinctes ;
- sockets des processus attachés à `ns-test`.

Namespace réseau (2/5) : créer et observer

```
# Créer deux environnements réseau isolés
ip netns add ns1
ip netns add ns2
# Observer ce que chacun voit
ip netns exec ns1 ip -br address
ip netns exec ns1 ip route
ip netns exec ns2 ip -br address
ip netns exec ns2 ip route
```

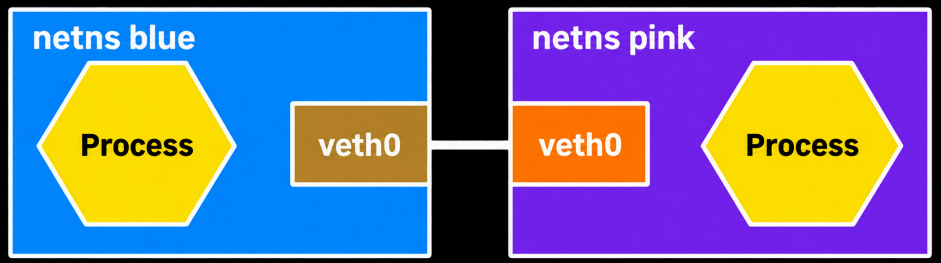
Lecture attendue

Les commandes sont identiques, mais leurs résultats dépendent du namespace d'exécution : interfaces, adresses et FIB peuvent être complètement différents.

Attention

Une interface réseau ne se trouve que dans un namespace à la fois.

Namespace réseau (3/5) : relier avec une paire `veth`



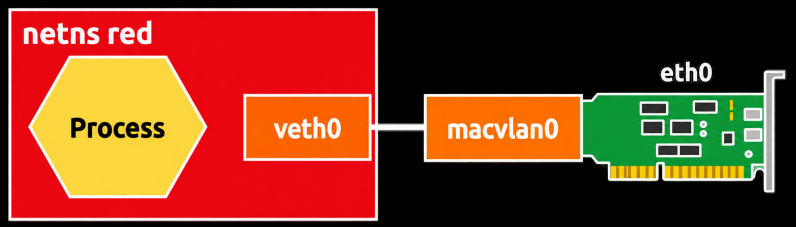
Câble virtuel

Un paquet injecté dans une extrémité ressort par l'autre. Chaque extrémité est déplacée dans son namespace et y devient une interface ordinaire.

Conséquence

Chaque namespace transmet ensuite selon sa propre FIB : la paire `veth` fournit le lien, pas la décision de routage.

Namespace réseau (4/5) : raccorder au réseau physique

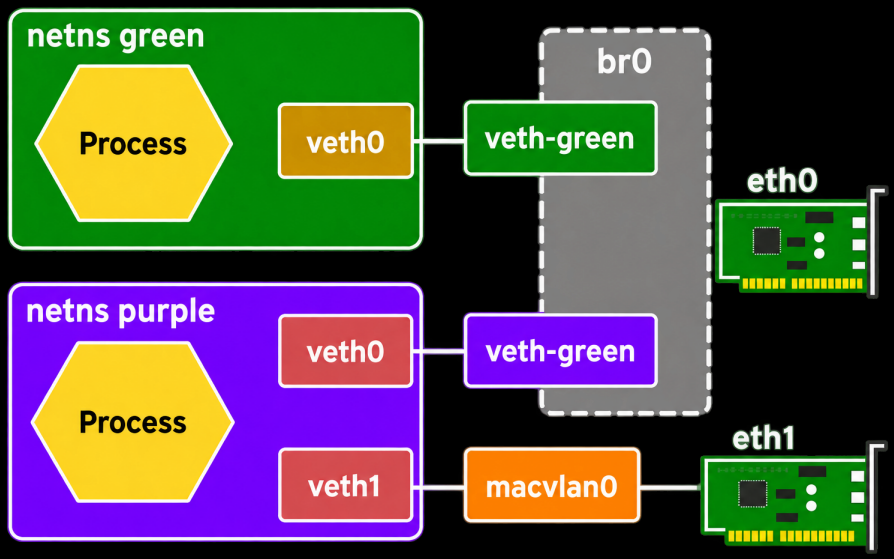


À comprendre

- Le namespace conserve interfaces, adresses et routes.
- `macvlan0` s'appuie directement sur l'interface physique parente.
- Aucune paire `veth` n'est nécessaire.
- Diagnostic : `ip netns exec`.

Le namespace atteint le réseau physique sans perdre son contexte réseau.

Namespace réseau (5/5) : relier via un bridge local



À comprendre

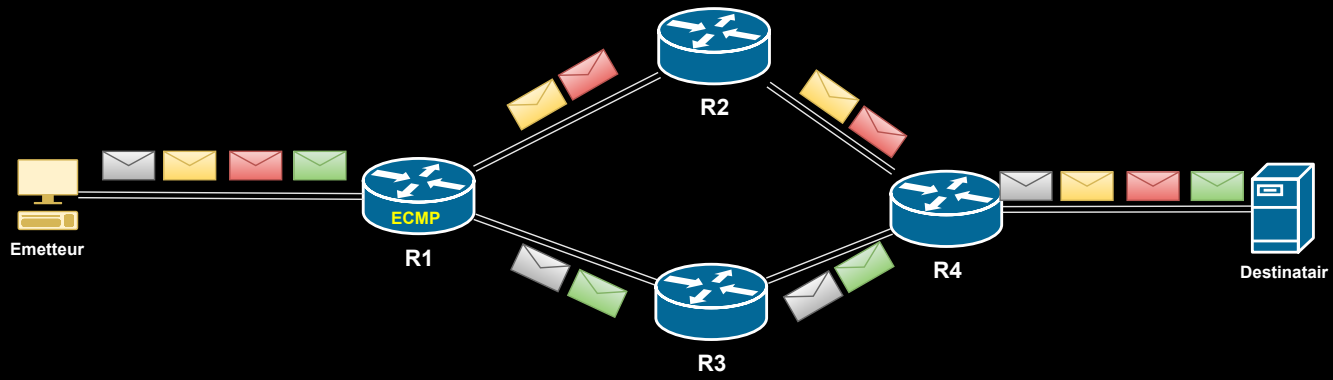
- Le bridge commute les trames comme un switch logiciel L2.
- Chaque `veth` relie une interface du namespace à une extrémité côté hôte raccordée à `br0`.
- Une interface physique peut prolonger ce segment L2.

Le bridge mutualise le domaine L2 ; chaque namespace conserve sa FIB.

ECMP : principe

Principe

Le **multipath routing** permet d'utiliser plusieurs chemins entre une source et une destination.



ECMP : Equal-Cost Multi-Path

- Plusieurs routes ayant le **même coût** sont installées dans la table.
- Le noyau répartit le trafic entre ces chemins.

ECMP : bénéfices et limites

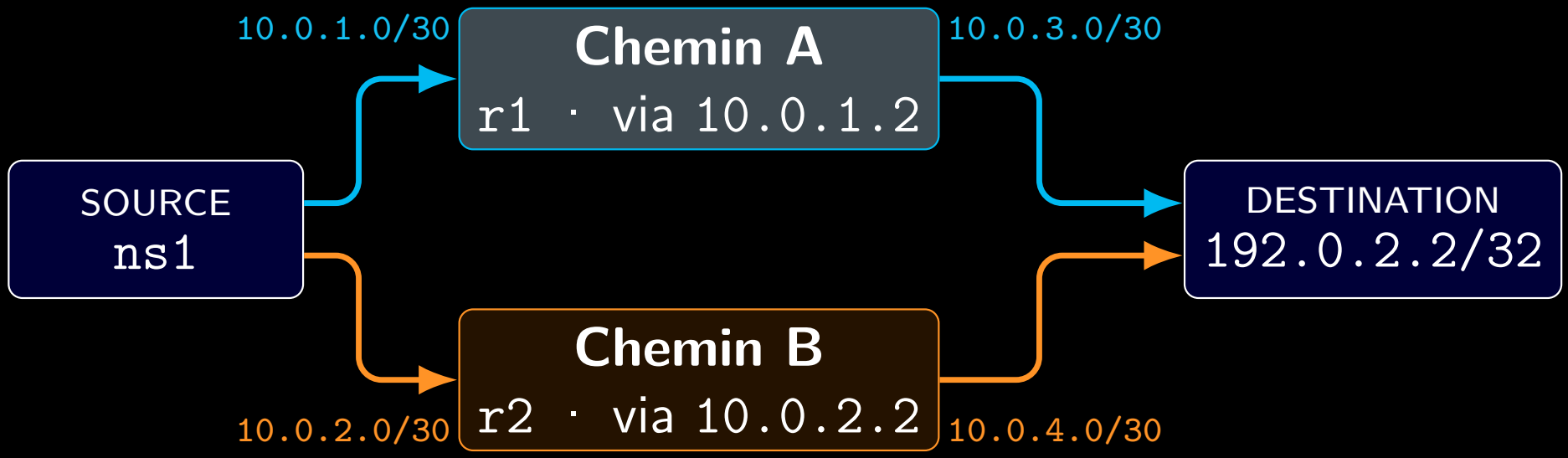
Bénéfices

- **Répartition de charge** (load balancing).
- **Résilience** : un chemin alternatif existe immédiatement en cas de panne.
- **Optimisation des ressources** réseau.

Limites

- Pas de prise en compte des différences de capacité entre liens.
- Hashing basé sur 5-tuple (src/dst IP, ports, protocole) $\Rightarrow$ peut créer des déséquilibres.

ECMP (1/4) : une destination, deux chemins



Principe ECMP

Une destination, une route, deux nexthops de même poids.

ECMP (2/4) : exprimer la route multipath

```
# Ici on  crée ns1, ns2, r1, r2, les veth et l'adressage.
# ns2 porte aussi 192.0.2.2/32, et r1/r2 savent le joindre.
ip netns exec r1 sysctl -w net.ipv4.ip_forward=1
ip netns exec r2 sysctl -w net.ipv4.ip_forward=1

# Une seule route, deux nexthops de coût équivalent
ip netns exec ns1 ip route add 192.0.2.2/32 \
  nexthop via 10.0.1.2 dev veth-ns1-r1 \
  nexthop via 10.0.2.2 dev veth-ns1-r2

ip netns exec ns1 ip route show 192.0.2.2/32
```

Attention

Les deux lignes `nexthop` doivent se lire sur le graphe : `10.0.1.2` mène à `r1`, `10.0.2.2` mène à `r2`.

ECMP (3/4) : observer la décision

```
# Voir la route multipath installée
ip netns exec ns1 ip route show 192.0.2.2/32

# Tester plusieurs sources ou plusieurs flux
ip netns exec ns1 ip route get 192.0.2.2 from 10.0.1.1
ip netns exec ns1 ip route get 192.0.2.2 from 10.0.2.1

# Générer plusieurs flux TCP
ip netns exec ns2 iperf3 -s -B 192.0.2.2
ip netns exec ns1 iperf3 -c 192.0.2.2 -P 20
```

Ce que l'on cherche

Vérifier si plusieurs flux utilisent les deux chemins, puis relier le résultat au hash ECMP plutôt qu'à un ordre de lignes dans la table.

ECMP (4/4) : limites à garder en tête

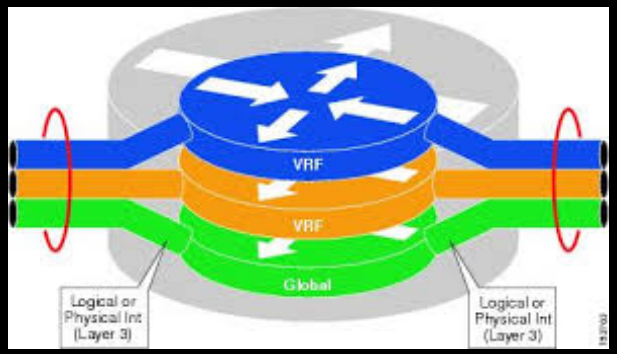
- Répartition **par flux**, selon un hash.
- Quelques gros flux peuvent déséquilibrer les chemins.
- Une reconvergence peut perdre des paquets.
- NAT, `conntrack` et stateful exigent souvent la symétrie.

❓ Aller par `r1`, retour par `r2` : quel état manque ?

VRF : une table de routage dédiée

Définition

Un **VRF** est une instance de table de routage séparée dans le même routeur ou hôte Linux.



Source : https://interpip.es/linux/creating-a-vrf-and-running-services-inside-it-on-linux/

Objectif

- Segmentation du trafic réseau sans matériel supplémentaire.
- Isolation logique entre clients, applications ou environnements.
- Similaire à des **VPN de niveau 3**.

VRF : cas d'usage

Cas d'usage

- Multi-tenancy (hébergement de plusieurs clients).
- Séparation trafic production / test.
- Opérateurs réseaux : routage client indépendant.

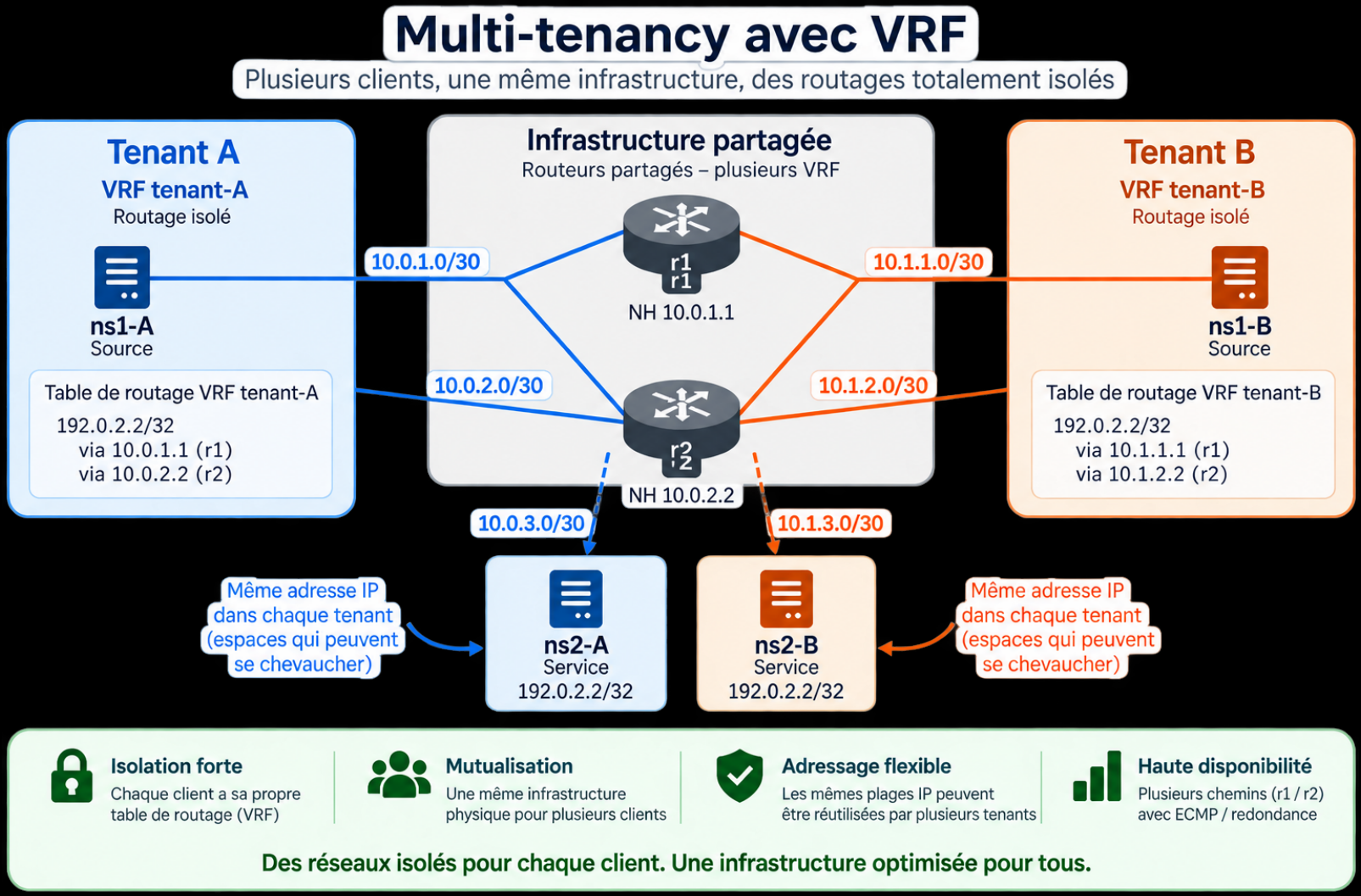
Sous Linux

Depuis le noyau 4.x, les VRFs sont pris en charge nativement :

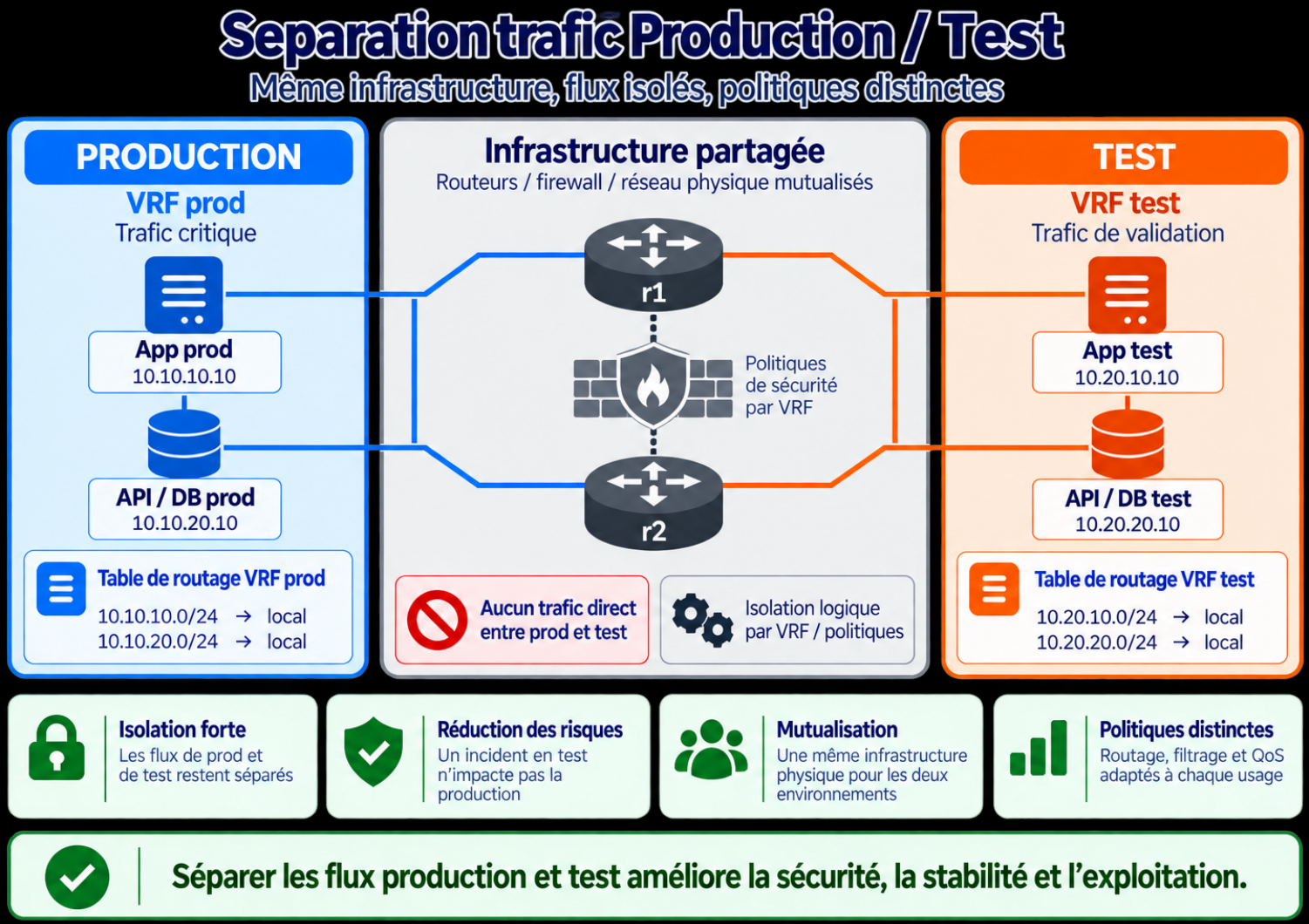
```
ip link add vrf-red type vrf table 100
```

Pour aller loin: https://thelinuxchannel.org/2023/10/vrf-virtual-routing-and-forwarding-introduction/

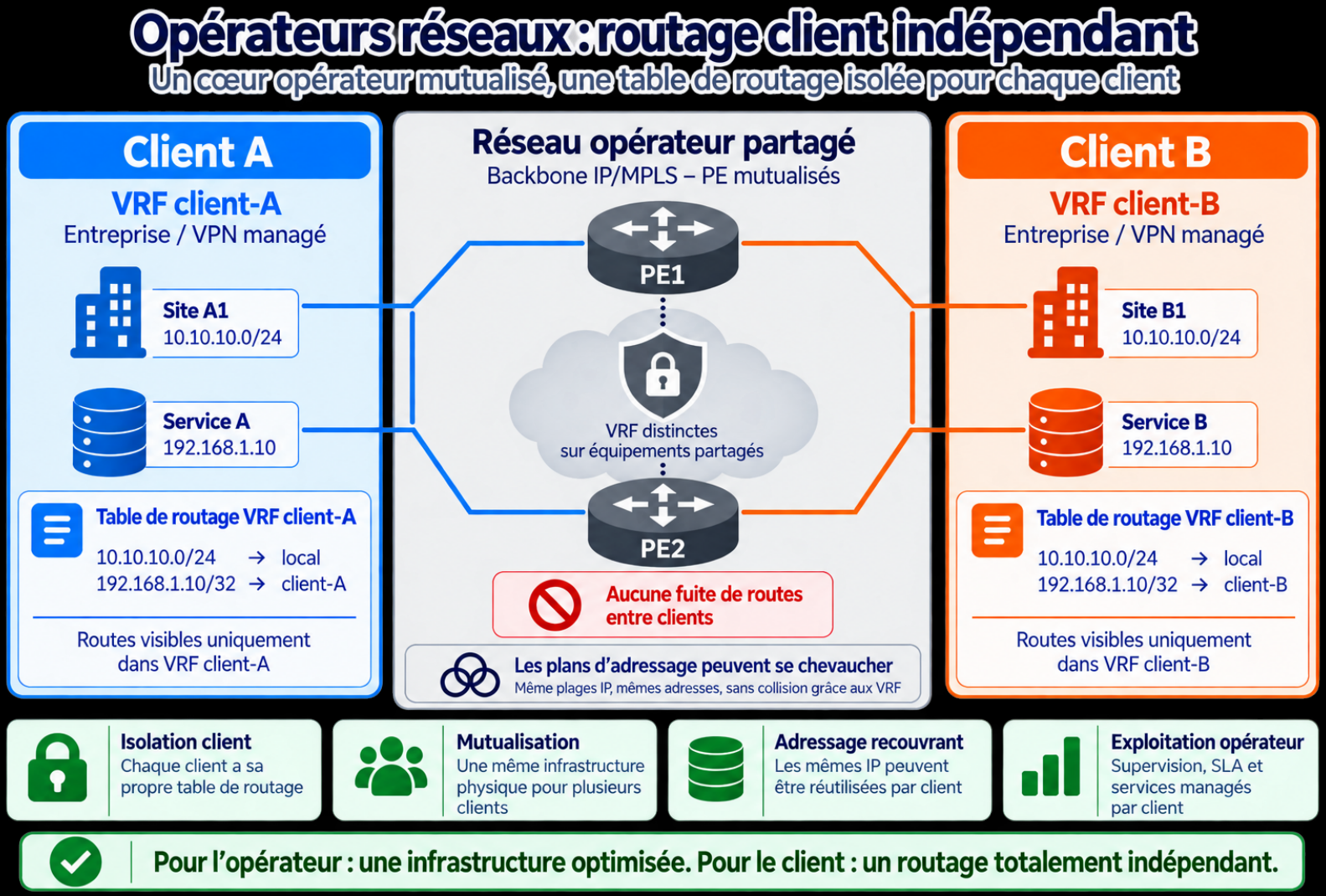
Multi-tenancy: hébergement clients



Séparation trafic production / test



routage client indépendant



VRF ou network namespace ?

VRF

- plusieurs tables de routage ;
- même pile réseau globale ;
- isolation centrée sur L3.

Network namespace

- piles réseau séparées ;
- interfaces, routes et Netfilter isolés ;
- sockets liées au namespace.

Choix d'architecture

VRF pour le routage ; namespace pour isoler toute la pile.

Interfaces tunnel et routage Linux

Point de vue du dataplane

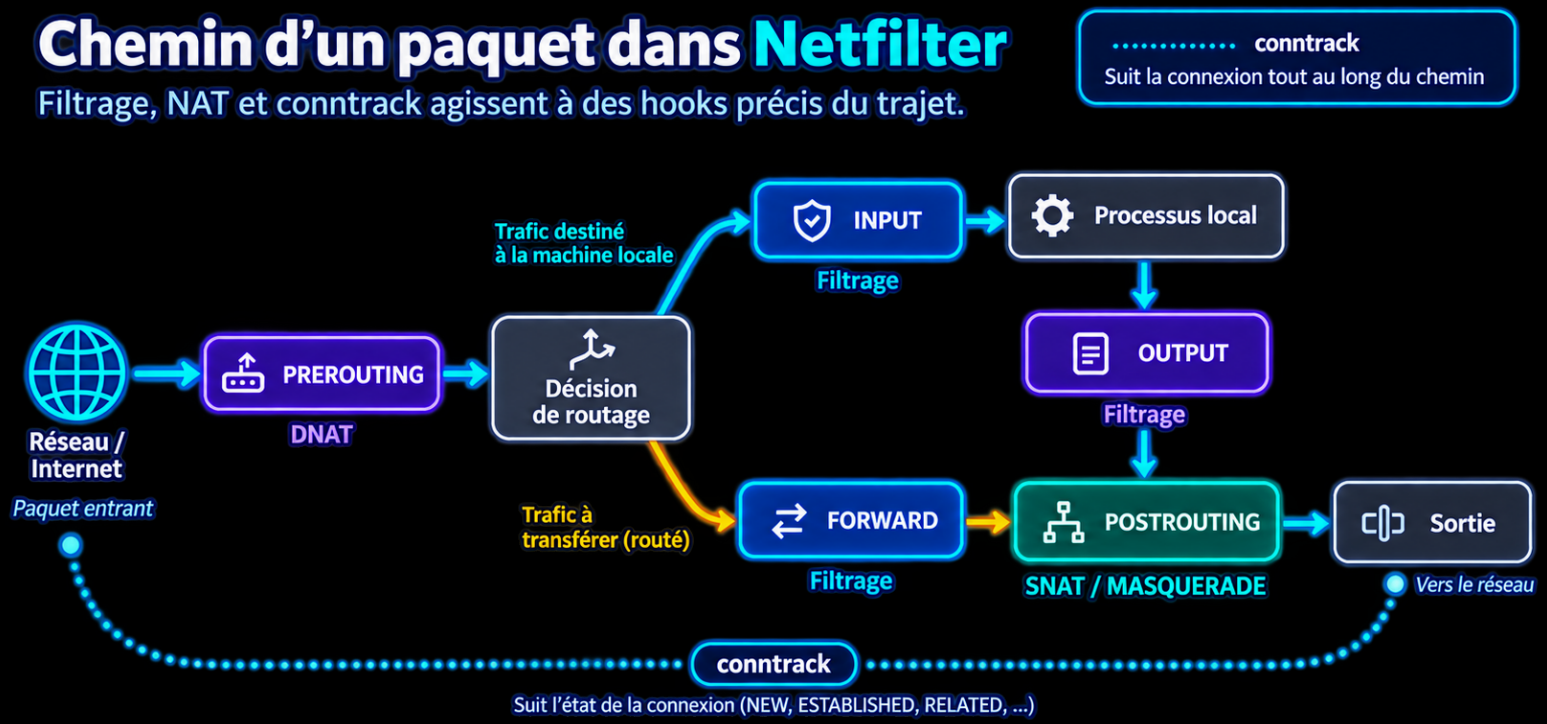
Une interface tunnel participe au routage comme une autre interface : la FIB sélectionne le préfixe, le nexthop et l'interface de sortie.

- Le tunnel ajoute ensuite une encapsulation au paquet sélectionné.
- Les notions d'underlay, d'overlay, de MTU et de sécurité sont étudiées dans le module 4.

Plan

5. Policy enforcement

Filtrage Linux - chemin paquet



Le filtrage, le NAT et conntrack ne sont pas des commandes isolées. Ils interviennent à des hooks précis du chemin paquet.

Conntrack - état des flux

- `NEW` : premier paquet observé pour un flux.
- `ESTABLISHED` : paquet appartenant à un flux déjà connu.
- `RELATED` : flux lié à une connexion existante.
- `INVALID` : paquet incohérent ou non associable à un état.

❓ Pourquoi un routage asymétrique peut-il perturber un pare-feu stateful ou une règle NAT ?

Policy enforcement : rôle architectural

Où placer un contrôle réellement traversé par le flux ?

- À la frontière entre zones de confiance.
- Sur le chemin réel du paquet.
- Avec un état cohérent et des logs exploitables.

Choisir le niveau de contrôle

L3/L4 stateless Filtrer adresses et ports.

Stateful Reconnaître les retours d'un flux.

NAT Publier, masquer ou raccorder.

Proxy L7 Contrôler application ou identité.

Journalisation Prouver et diagnostiquer.

Compromis

Un contrôle plus riche ajoute état, latence et dépendances.

Netfilter et routage : ce que cela ajoute

Routage

- choisit un **chemin** ;
- consulte la FIB et les règles ;
- produit un nexthop et une interface ;
- reste essentiellement **stateless**.

Netfilter / nftables

- applique une **politique** ;
- filtre selon interface, adresse, port, état ;
- fait NAT avant/après le forwarding ;
- expose l'état via `conntrack`.

```
paquet -> hooks Netfilter -> décision FIB
       -> hooks Netfilter -> sortie
```

Message architectural

La route choisit le chemin ; Netfilter autorise, bloque ou traduit le paquet.

Lire une policy existante

Ce que l'on cherche

- Quelles zones peuvent communiquer ?
- Trafic local, sortant ou traversant ?
- NAT avant ou après le forwarding ?
- Dépendance à `conntrack` ?

Compétence attendue

Expliquer l'architecture réellement appliquée.

De iptables à nftables

Pourquoi en parler ?

- `iptables` reste présent dans l'existant.
- `nftables` est la cible moderne.
- Lire les deux permet audit et migration.

Ce que change nftables

- modèle unifié IPv4, IPv6, ARP et bridge ;
- sets, maps et règles dynamiques ;
- automatisation et validation facilitées.

nftables - filtrage stateful

```
nft add table inet fw
nft 'add chain inet fw input { type filter hook input priority 0; policy drop; }'

nft add rule inet fw input iif lo accept
nft add rule inet fw input ct state established,related accept
nft add rule inet fw input ct state invalid drop
nft add rule inet fw input tcp dport { 22, 80, 443 } accept
```

Lecture

La politique restrictive est portée par la chaîne, et les règles expriment les états conntrack directement avec `ct state`.

NAT dans iptables et nftables

Concepts clés

- **SNAT** : modifier la source ;
- **DNAT** : modifier la destination ;
- **MASQUERADE** : SNAT adapté à une adresse dynamique.

Point d'architecture

Le NAT n'est pas appliqué au même moment selon le besoin : le DNAT modifie la destination avant le routage, tandis que le SNAT modifie la source avant la sortie.
Les adresses observées dans les règles et les captures dépendent donc du point d'observation.

nftables - NAT sortant

```
nft add table ip nat
nft 'add chain ip nat postrouting { type nat hook postrouting priority srcnat; }'

# LAN privé sortant vers Internet IPv4
nft add rule ip nat postrouting ip saddr 10.10.10.0/24 oif eth0 masquerade

# Diagnostic
nft list ruleset
conntrack -L
```

❓ Que devient l'état conntrack si la route retour ne repasse pas par ce routeur ?

Synthèse : exprimer et vérifier une policy Linux

Existant

iptables reste fréquent : il faut savoir identifier ses tables, ses chaînes et l'intention des règles.

Cible moderne

nftables unifie IPv4/IPv6, les hooks, les sets et les règles stateful dans un modèle plus industrialisable.

Méthode commune

Identifier le chemin réel, localiser le hook, lire la policy, observer l'état et produire une preuve avec `nft`, `conntrack` et `tcpdump`.

Exercice de décision

Situation

Trois zones partagent un pare-feu Linux pour l'accès Internet.

- Choisir le mécanisme d'isolation.
- Placer NAT et filtrage sans casser la symétrie.
- Définir les preuves du chemin réel.

Preuve attendue

Schéma, règles de forwarding, route, policy et état.

Contribution au projet fil rouge

Décision d'architecture

Matérialiser zones et politiques sur un dataplane Linux.

- associer zones, interfaces, piles et tables ;
- placer Netfilter, NAT et l'état ;
- prouver chemin, symétrie et policy.

Livrable

M2 – Design dataplane et preuves de forwarding.

Références bibliographiques (1/2)

Documentation officielle

- Netfilter Project : `https://www.netfilter.org/`
- Manuels Linux : `man iptables`, `man nft`, `man ip`
- Documentation kernel Linux (networking) : `https://www.kernel.org/doc/html/latest/networking/index.html`

Guides pratiques

- *Linux Firewalls: Attack Detection and Response with iptables, psad, and fwsnort*, Michael Rash, No Starch Press.
- ArchWiki – iptables : `https://wiki.archlinux.org/title/iptables`
- ArchWiki – nftables : `https://wiki.archlinux.org/title/nftables`

Références bibliographiques (2/2)

Guides complémentaires et articles

- Debian Wiki – nftables : `https://wiki.debian.org/nftables`
- P. Ayuso, *Mastering Linux Security and Hardening*, Packt, 2022.
- Blog RedHat – nftables vs iptables : `https://www.redhat.com/en/blog/iptables-nftables`